

Case Swarm Chat Bridge — Real-Time Messaging Options

Salesforce ↔ Microsoft Teams Case Swarm Chat — architecture options considered for keeping chat messages flowing live between an agent in Salesforce and participants in a Teams chat, without exhausting platform limits on either side.

The requirement: a Salesforce user can chat with a Teams chat tied to a Case, live in both directions, without the read side polling Microsoft Graph or Salesforce's API so aggressively that it trips a platform limit.

The core tension: showing "new messages just arrived" requires either *polling* (ask repeatedly, costs something proportional to frequency) or a true *push* channel (something notifies you the moment it happens, near-zero cost regardless of frequency). Every option below is a different way of resolving that tension, and each has failed or succeeded against a different limit.

Option 1 — Apex polls Microsoft Graph directly (original design)

The Case Swarm Chat LWC calls a Salesforce Apex method (`syncMessages`) on a timer; Apex calls Microsoft Graph (`GET /chats/{id}/messages`) via a Named Credential, and sends via the same pattern.

Aspect	Detail
Read mechanism	LWC → Apex (every 5s) → Graph GET
Send mechanism	LWC → Apex → Bot Framework Connector
Limit exposure	Graph 10 req/10s shared per app/tenant across every open widget, <i>plus</i> Salesforce Apex callout count per transaction
Implementation cost	Lowest — no Azure Functions needed at all
Latency	Up to poll interval
Verdict	Simple, but every read is an Apex callout, and Graph's throttle is shared with every other integration touching the same app registration.

Option 2 — Azure Functions relay/bridge (built and deployed this session)

An Azure Function App (`teamsBotRelay` , `chatHistory` , `chatSend`) sits between Salesforce and Microsoft. The relay acks plain Teams chat messages without forwarding them to Salesforce (avoiding Salesforce's Daily API Request allocation). The LWC polls the Azure `chatHistory` endpoint directly, which calls Graph on Salesforce's behalf; sending goes through Azure `chatSend` , which calls the Bot Framework Connector. A signed HMAC session token (minted by Apex, verified by Azure) gates access.

Aspect	Detail
Read mechanism	LWC → Azure <code>chatHistory</code> (every 5s, was 2s) → Graph GET
Send mechanism	LWC → Azure <code>chatSend</code> → Bot Framework Connector
Limit exposure	Graph 10 req/10s — hit live in testing ("API calls quota exceeded! 10Per10Secs"); mitigated by raising the poll interval to 5s and adding 429 backoff, not eliminated
Salesforce impact	None for messaging — relay acks <code>message</code> -type activities instead of forwarding, so no Daily API Request or Apex callout per chat message
Implementation cost	Highest so far — new Function App, HMAC token bridge, CORS/CSP configuration, Azure AD app credentials for both Graph and Bot Framework
Latency	Up to poll interval (5s)
Status	Currently deployed and working , with the Graph throttle as its one known soft spot

Option 3 — Azure Table Storage cache (proposed, not yet built)

Instead of discarding plain Teams messages, the relay captures their content (already present in the Bot Framework activity payload — no Graph call needed) and writes it to an Azure Table Storage row. `chatHistory` reads from that Table instead of calling Graph. Everything else (send path, HMAC token bridge, LWC poll loop) stays unchanged.

Aspect	Detail
Read mechanism	LWC → Azure <code>chatHistory</code> (poll) → Table Storage read (no Graph call)
Send mechanism	Unchanged — Azure <code>chatSend</code> → Bot Framework Connector
Limit exposure	Graph throttle eliminated entirely — Table Storage's own ceiling is tens of thousands of transactions/sec per account, categorically larger headroom than any other option here
Salesforce impact	None — unchanged from Option 2

Cost	Negligible — roughly \$0.00002 per 45-minute/60-message session; can typically reuse the Storage Account the Function App already has for its own bookkeeping
Implementation cost	Smallest change from what's already deployed — only <code>teamsBotRelay.js</code> (write path) and <code>chatHistory.js</code> (read path) change; everything else (send, token auth, LWC poll loop, usage tracking, reset flow) stays exactly as-is
Trade-off	Reintroduces a small amount of server-side persistence in Azure — a step away from the original "Teams is the only source of truth, nothing persisted" principle, though the cached rows are short-lived, not a permanent transcript
Latency	Up to poll interval — can safely be tightened below 5s now, since Graph is no longer in the read path at all

Option 4 — Simple webhook / Platform Event push (no persistence anywhere)

Instead of the relay discarding `message`-type Bot Framework activities, it forwards them to Salesforce as before. `TeamsBotMessagingResource` extracts the text/sender directly from the activity payload and publishes a Salesforce Platform Event — no database row anywhere, Salesforce or Azure. The LWC subscribes via `empApi` and displays messages the instant they're published. Sending reverts to the original direct Apex → Bot Framework Connector call, since outbound sends never needed Azure in the first place.

Aspect	Detail
Read mechanism	Bot Framework push → relay forwards → Apex publishes Platform Event → LWC <code>empApi</code> subscribe (true push, zero polling)
Send mechanism	Reverts to direct Apex → Bot Framework Connector (Azure <code>chatSend</code> no longer needed)
Limit exposure	Graph: zero calls, ever for messaging. Salesforce Daily API Request: 1 per Teams-originated message (the forwarded activity). Platform Event daily allocation: 1 per Teams-originated message. Both bounded by human typing speed, not an artificial poll interval — comfortably distant from any realistic daily ceiling at this app's scale, but a real, finite number, not zero.
Persistence	None anywhere — no <code>Swarm_Message__c</code> , no Table Storage. Trade-off: no chat history on tab reopen unless a one-time (non-recurring) Graph fetch is added for initial backfill.
Implementation cost	Largest change of any option relative to what's currently deployed — retires <code>chatHistory.js</code> , <code>chatSend.js</code> , <code>botSend.js</code> , <code>graphChat.js</code> , and the HMAC token bridge; adds new Apex Platform Event handling and new LWC <code>empApi</code> subscription code
Latency	Instant — genuine push, no poll interval at all

Option 5 — Azure SignalR Service

A managed WebSocket push service. The relay broadcasts new message content directly to browser clients connected to a SignalR group scoped to the chat, bypassing Salesforce's API surface entirely for message delivery. The browser holds a persistent connection instead of polling anything.

Aspect	Detail
Read mechanism	Bot Framework push → relay broadcasts via SignalR → browser WebSocket (true push, zero Salesforce involvement)
Send mechanism	Unaffected either way — SignalR only solves delivery to the browser
Limit exposure	Graph: zero calls Salesforce: zero impact SignalR Free tier: 20 concurrent connections, 20,000 messages/day — the concurrent-connection cap could bind sooner in absolute terms than Platform Events' daily allocation if usage grows past ~20 simultaneously open chat widgets org-wide.
Cost	Free tier likely sufficient at current scale; paid tier (~\$1-2/unit/day plus overages) only needed at meaningfully larger scale
Implementation cost	Highest of all options — new Azure resource, a "negotiate" endpoint for scoped connection tokens, a client-side SignalR JS library loaded via Salesforce Static Resource (LWC has no native npm import), a new CSP Trusted Site entry for the WebSocket connection, and connection lifecycle/reconnect handling
Latency	Instant
Verdict	Architecturally the "purest" real-time design, but solves a Salesforce-involvement problem this app hasn't actually demonstrated a need to solve, at the highest implementation cost of any option considered.

Side-by-side comparison

Option	Kills Graph throttle?	Touches Salesforce limits?	New infrastructure	Change from current deployment	Latency
1. Apex → Graph (original)	No	Apex callouts only	None	N/A — superseded	Up to poll interval
2. Azure Bridge (current)	Mitigated, not eliminated	None for messaging	Already built	None — already deployed	Up to 5s
3. Table Storage	Eliminated	None	Reuses existing Storage Account	Smallest	Up to poll interval (can be tightened)
4. Platform Event push	Eliminated	1 Daily API Request + 1 event per Teams message	None new	Largest — retires most of Option 2	Instant
5. SignalR	Eliminated	None	SignalR service + negotiate function + client library + CSP entry	Largest of all	Instant

Recommendation

Table Storage (Option 3) is the best fit for this app's actual demonstrated requirements: a handful of concurrent case-swarm chats, human-typing-bound message volume, and one limit that has actually failed in production testing (Graph's 10-requests/10-seconds throttle). It eliminates that specific failure completely, costs a few cents a month, and — critically — requires the smallest possible change to a system that is already built, deployed, and mostly working: only the relay's write path and the history endpoint's read path change; the send path, HMAC token bridge, LWC polling loop, usage tracking, and reset flow all stay exactly as they are today.

Platform Events and SignalR are both legitimate, more "architecturally pure" real-time designs — genuine push instead of polling — but they solve problems this app hasn't actually hit (Salesforce API/Platform Event volume has never been the bottleneck; Graph

has). They also cost meaningfully more to build: Platform Events require unwinding several already-debugged Azure components, and SignalR adds an entirely new managed service, client library, and connection-auth layer. Either is worth revisiting if this feature's usage grows substantially beyond a handful of simultaneous chats, or if a future requirement specifically calls for zero Salesforce involvement in message delivery — neither of which is true today.

Document generated from the architecture discussion during the Case Swarm Chat Bridge build-out. Reflects the state of the implementation and testing as of this writing.